Exp1: Program

A. Registration Page — register.html

```
<html>
<head>
   <title>Registration Page</title>
</head>
<body>
   <h1>User Registration</h1>

   <form action="login.html" method="get">
      <label>Full Name:</label>
      <input type="text" name="name" required>
      <br><br>
      <label>Email:</label>
      <input type="email" name="email" required>
      <br><br>
      <label>Password:</label>
      <input type="password" name="password"
           minlength="6" required>
      <br><br>

      <label>Confirm Password:</label>
      <input type="password" name="confirm"
           minlength="6" required>
      <br><br>
      <input type="submit" value="Register">
      <input type="reset" value="Clear">
   </form>

   <p>Already registered?
      <a href="login.html">Login Here</a>
   </p>
</body>
</html>
```

B. Login Page — login.html

```
<html>
<head>
   <title>Login Page</title>
</head>
<body>
   <h1>User Login</h1>

   <form action="success.html" method="get">
      <label>Email:</label>
      <input type="email" name="email" required>
      <br><br>
```

```
    <label>Password:</label>
    <input type="password" name="password"
        required>
    <br><br>

    <input type="submit" value="Login">
    <input type="reset" value="Clear">
  </form>

  <p>New user?
    <a href="register.html">Register Here</a>
  </p>
</body>
</html>
```

C. Success Page — success.html

```
<html>
<head>
  <title>Success Page</title>
</head>
<body>
  <h1>Login Successful!</h1>

  <p>Welcome to the application.</p>
  <p>You have successfully reached the home page.</p>

  <a href="login.html">Logout</a>
</body>
</html>
```

Git Commands to Upload the Project

Step 1: Open Command Prompt in the **24681A0501** project directory.
Step 2: Initialize the repository.
git init
Step 3: Add the HTML files.
git add .
Step 4: Commit the changes.
git commit -m "Create HTML registration and login pages"
Step 5: Connect the GitHub repository.
First, create an empty repository named exp1 under the cjits25 organization.

- ✓ git config --global user.name "cjits25"
- ✓ git config --global user.email cjitsjn@gmail.com
- ✓ git remote add origin **https://cjits25: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@github.com/cjits25/24681A0501**
- ✓ git branch exp1
- ✓ git push -u origin exp1

Verify the uploaded files at: **https://github.com/cjits25/24681A0501 in exp1 branch.**

Exp2: Program

Create a Simple Java Application Let's create a basic Java app that prints "Hello, World!" and performs a simple addition.

Create a exp2 folder in your RollNumber folder.

Execute following git commands:

- ✓ git init
- ✓ git add .
- ✓ git commit –m "Exp2 progrm"
- ✓ git config --global user.name "cjits25"
- ✓ git config --global user.email cjitsjn@gmail.com
- ✓ git remote add origin **https://cjits25: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@github.com/cjits25/24681A0501**
- ✓ git branch exp2
- ✓ git push –u origin exp2

Main.java :

```
public class Main {
   public static void main(String[] args)
{
      System.out.println("Hello, India!");
      int a = 5, b = 70;
      int sum = a + b;
      System.out.println("Sum: " + sum);
   }
}
```

build.bat :

```
echo Building Java App...
mkdir out 2>nul
javac -d out src\Main.java

echo Running Java App...
cd out
java Main

echo Build and run completed.
```

Open the chrome browser and type in the address: https://localhost:8080

Jenkins Freestyle project, configure Git to download this repository and build the files from the exp2 branch.

1. Configure Jenkins → Source Code Management

Open Jenkins → Exp2-Build → Configure and set these values:

Setting	Value
Project type	Freestyle project
Source Code Management	Git

Repository URL	https://github.com/cjits25/24681A0501.git
Credentials	None if the repository is public
Branch Specifier	*/exp2

2. Configure the Windows batch build

Under Build Steps, click Add build step → Execute Windows batch command and type file name:

build.bat

Click Save → Build Now → Console Output.

OutPut:

```
C:\ProgramData\Jenkins\.jenkins\workspace\exp2\out>java Main

Hello, India!

Sum: 75

C:\ProgramData\Jenkins\.jenkins\workspace\exp2\out>echo Build and run completed.

Build and run completed.

Finished: SUCCESS
```

Exp3 Program

- ✓ Containerization Using Docker:
- ✓ Create a exp3 folder in your RollNumber folder:
  ex: D:\24681A0501\exp3
- ✓ In this folder create a templates folder
- ✓ In this folder create two html(register.html and success.html files).
- ✓ Create a app.py folder in the exp3 folder.

register.html:

```
<html>
<head>
 <title>Register</title>
</head>
<body>
 <h1>Register</h1>
 <form method="POST">
 <input type="text" name="username" placeholder="Username"
required><br>
 <input type="email" name="email" placeholder="Email"
required><br>
 <input type="password" name="password"
placeholder="Password" required><br>
 <button type="submit">Register</button>
 </form>
</body>
</html>
```

success.html:

```
<html>
<head>
 <title>Success</title>
</head>
<body>
 <h1>Registration Successful!</h1>
 <p>Thank you for registering.</p>
</body>
</html>
```

app.py :

```
from flask import Flask, render_template, request, redirect, url_for
app = Flask(__name__)
# Temporary storage (replace with a database in production)
users = []
@app.route('/')
def home():
    return redirect(url_for('register'))
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        email = request.form['email']
        password = request.form['password']

        users.append({'username': username, 'email': email})
        return redirect(url_for('success'))
```

```
    return render_template('register.html')

@app.route('/success')
def success():
    return render_template('success.html')

if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=True)
```

Dockerfile:

```
FROM python:3.9-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

requirements.txt :

```
Flask==2.3.2
```

- ✓ Now start the Docker Engine: select Docker Desktop from Windows menu(click the App once only).
- ✓ After successfully started Docker go to the exp3 folder at address bar type cmd and type below commands.

D:\24681A0501\exp3>docker build –t exp3 .

D:\24681A0501\exp3>docker run –p 5000:5000 exp3

If these commands sucssfully executes open the chrome browser and type at address bar:

http://localhost:5000

Exp4 Program

Node.js Programming and Selenium Testing

- ✓ Create a 501exp4 folder in the C drive at Xampp's htdocs folder
- ✓ ex: C:\xampp\htdocs/501exp4
- ✓ Open the Xampp Control Panel and start Apache Service.
- ✓ Go to 501exp4 and create a **login.html** page
- ✓ And create **testLogin.js** Script file for Test Cases to check login page.

login.html:

```
<html>
<head>
  <title>Login Page</title>
</head>
<body>
  <h2>Login Form</h2>
  <form id="loginForm">
    <label>Username:</label>
    <input type="text" id="username"><br><br>
    <label>Password:</label>
```

```
  <input type="password" id="password"><br><br>
  <button type="button" id="loginButton">Login</button>
 </form>

 <p id="message"></p>

 <script>
  document.getElementById("loginButton").addEventListener("click", function() {
   const user = document.getElementById("username").value;
   const pass = document.getElementById("password").value;
   if (user === "cjits" && pass === "12345") {
    document.getElementById("message").innerText = "Login successful!";
   } else {
    document.getElementById("message").innerText = "Invalid credentials!";
   }
  });
 </script>
</body>
</html>
```

testLogin.js:

```
const { Builder, By, until } = require('selenium-webdriver');
(async function loginTest() {
 let driver = await new Builder().forBrowser('chrome').build();
 try {
  await driver.get('http://localhost/exp11/login.html');
  await driver.sleep(1000);
  await driver.findElement(By.id('username')).sendKeys('cjits');
  await driver.findElement(By.id('password')).sendKeys('12345');
  await driver.findElement(By.id('loginButton')).click();
  let message = await driver.wait(until.elementLocated(By.id('message')), 2000);
  let text = await message.getText();
  console.log("Message displayed:", text);
  if (text === "Login successful!") {
   console.log("✅Test Passed!");
  } else {
   console.log("❌Test Failed!");
  }

 } catch (err) {
  console.error("Test Error:", err);
 } finally {
  await driver.quit();
 }
})();
```

How to Test(Process ..): Goto C:\xampp\htdocs\501exp4 folder and at address bar type cmd and run below commands:

1. npm init -y
2. npm install selenium-webdriver chromedriver

After install drivers check the testlogin.js

3. node testLogin.js

Exp5 Program

Selenium Test Case Development for a Application

- ✓ Create a 501exp5 folder in the C drive at Xampp's htdocs folder
- ✓ ex: C:\xampp\htdocs/501exp5
- ✓ Open the Xampp Control Panel and start Apache Service.
- ✓ Go to 501exp5 and create a **login.php** page
- ✓ And create **test_login_php.py** python code file for Test Cases to check login page.

login.php:

```
<html>
<head>
  <title>Simple Login</title>
  <style>
    body {
      font-family: Arial;
      background: #f3f3f3;
    }
    .login-box {
      width: 300px;
      margin: 100px auto;
      background: white;
      padding: 20px;
      border-radius: 10px;
      box-shadow: 0 0 10px gray;
    }
    input[type=text], input[type=password] {
      width: 100%;
      padding: 8px;
      margin: 8px 0;
      border: 1px solid #ccc;
      border-radius: 4px;
    }
    input[type=submit] {
      background-color: #4CAF50;
      color: white;
      padding: 10px;
      border: none;
      cursor: pointer;
      width: 100%;
      border-radius: 4px;
    }
    input[type=submit]:hover {
      background-color: #45a049;
    }
  </style>
</head>
<body>
<div class="login-box">
  <h2>Login Form</h2>
  <form method="POST">
```

```
        <label>Username:</label>
        <input type="text" name="username" required>
        <label>Password:</label>
        <input type="password" name="password" required>
        <input type="submit" name="login" value="Login">
    </form>

    <?php
    if (isset($_POST['login'])) {
        $username = $_POST['username'];
        $password = $_POST['password'];

        // Simple hardcoded check
        if ($username == "admin" && $password == "12345") {
            echo "<p style='color:green;'>Login Successful!</p>";
        } else {
            echo "<p style='color:red;'>Invalid Username or Password</p>";
        }
    }
    ?>
</div>
</body>
</html>
```

test_login_php.py :

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
import time

# ---------- CONFIGURATION ----------
URL = "http://localhost/exp12/login.php"   # change if hosted elsewhere
USERNAME = "admin"
PASSWORD = "12345"

# ---------- SETUP CHROME ----------
driver = webdriver.Chrome()
driver.maximize_window()
driver.get(URL)
time.sleep(2)

# ---------- TEST CASE 1: Page Title and URL ----------
print("TC01 - Checking page title and URL")
assert "Login" in driver.title, "❌Page title mismatch"
assert "login" in driver.current_url, "❌URL does not contain 'login'"
print("✅Page loaded correctly")

# ---------- TEST CASE 2: Successful Login ----------
print("TC02 - Valid login test")
driver.find_element(By.NAME, "username").clear()
driver.find_element(By.NAME, "username").send_keys(USERNAME)
```

```
driver.find_element(By.NAME, "password").clear()
driver.find_element(By.NAME, "password").send_keys(PASSWORD)
driver.find_element(By.NAME, "login").click()
time.sleep(1)

page_source = driver.page_source
assert "Login Successful" in page_source, "❌Valid login failed"
print("✅Valid login passed")

# ---------- Go back to login page ----------
driver.get(URL)
time.sleep(1)

# ---------- TEST CASE 3: Invalid Username ----------
print("TC03 - Invalid username test")
driver.find_element(By.NAME, "username").send_keys("wronguser")
driver.find_element(By.NAME, "password").send_keys(PASSWORD)
driver.find_element(By.NAME, "login").click()
time.sleep(1)
assert "Invalid" in driver.page_source, "❌Invalid username not handled"
print("✅Invalid username handled correctly")

# ---------- Go back to login page ----------
driver.get(URL)
time.sleep(1)

# ---------- TEST CASE 4: Invalid Password ----------
print("TC04 - Invalid password test")
driver.find_element(By.NAME, "username").send_keys(USERNAME)
driver.find_element(By.NAME, "password").send_keys("wrongpass")
driver.find_element(By.NAME, "login").click()
time.sleep(1)
assert "Invalid" in driver.page_source, "❌Invalid password not handled"
print("✅Invalid password handled correctly")

# ---------- Go back to login page ----------
driver.get(URL)
time.sleep(1)

# ---------- TEST CASE 5: Empty Fields ----------
print("TC05 - Empty fields test")
driver.find_element(By.NAME, "username").clear()
driver.find_element(By.NAME, "password").clear()
driver.find_element(By.NAME, "login").click()
time.sleep(1)
# Check for 'required' attribute behavior or no action
print("✅Empty fields test executed (browser shows required warning if HTML5 required
attribute is used)")

# ---------- TEST CASE 6: Verify Input Elements Exist ----------
print("TC06 - Checking presence of input fields")
```

```
username_field = driver.find_element(By.NAME, "username")
password_field = driver.find_element(By.NAME, "password")
login_button = driver.find_element(By.NAME, "login")

assert username_field.is_displayed(), "❌Username field missing"
assert password_field.is_displayed(), "❌Password field missing"
assert login_button.is_displayed(), "❌Login button missing"
print("✅All input fields found")

# ---------- CLEANUP ----------
time.sleep(2)
driver.quit()
print("\n All test cases executed successfully!")
```

How to Test(Process ..): Goto C:\xampp\htdocs\501exp5 folder and at address bar type cmd and run below commands:

1. pip install selenium
2. py test_login_php.py